

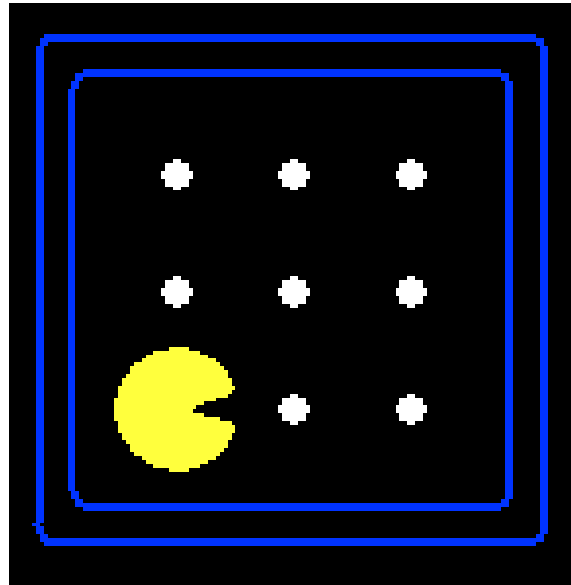
2. Uninformed Search and Heuristic Search

Outline

- Search Problems
- Uninformed Search
 - Depth-First Search
 - Breadth-First Search
 - Uniform-Cost Search
- Heuristic Search
 - Greedy
 - A*

Search Problems

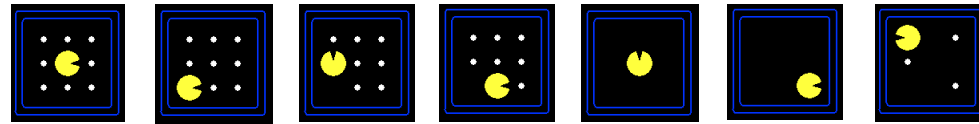
- Pacman (agent)
- states
- actions (function)
- goal test



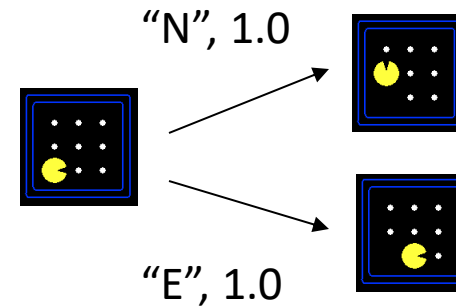
Search Problems

- A **search problem** consists of:

- A state space

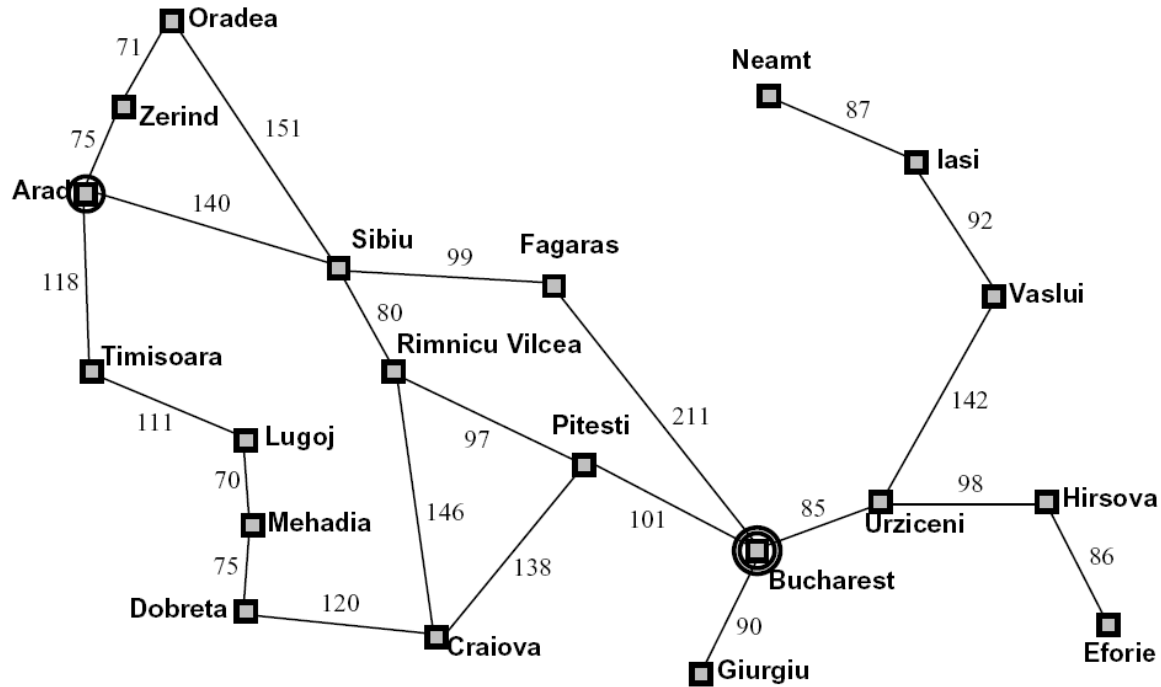


- A successor function
(with actions, costs)



- A start state and a goal test
- A **solution** is a sequence of actions (a plan) which transforms the start state to a goal state

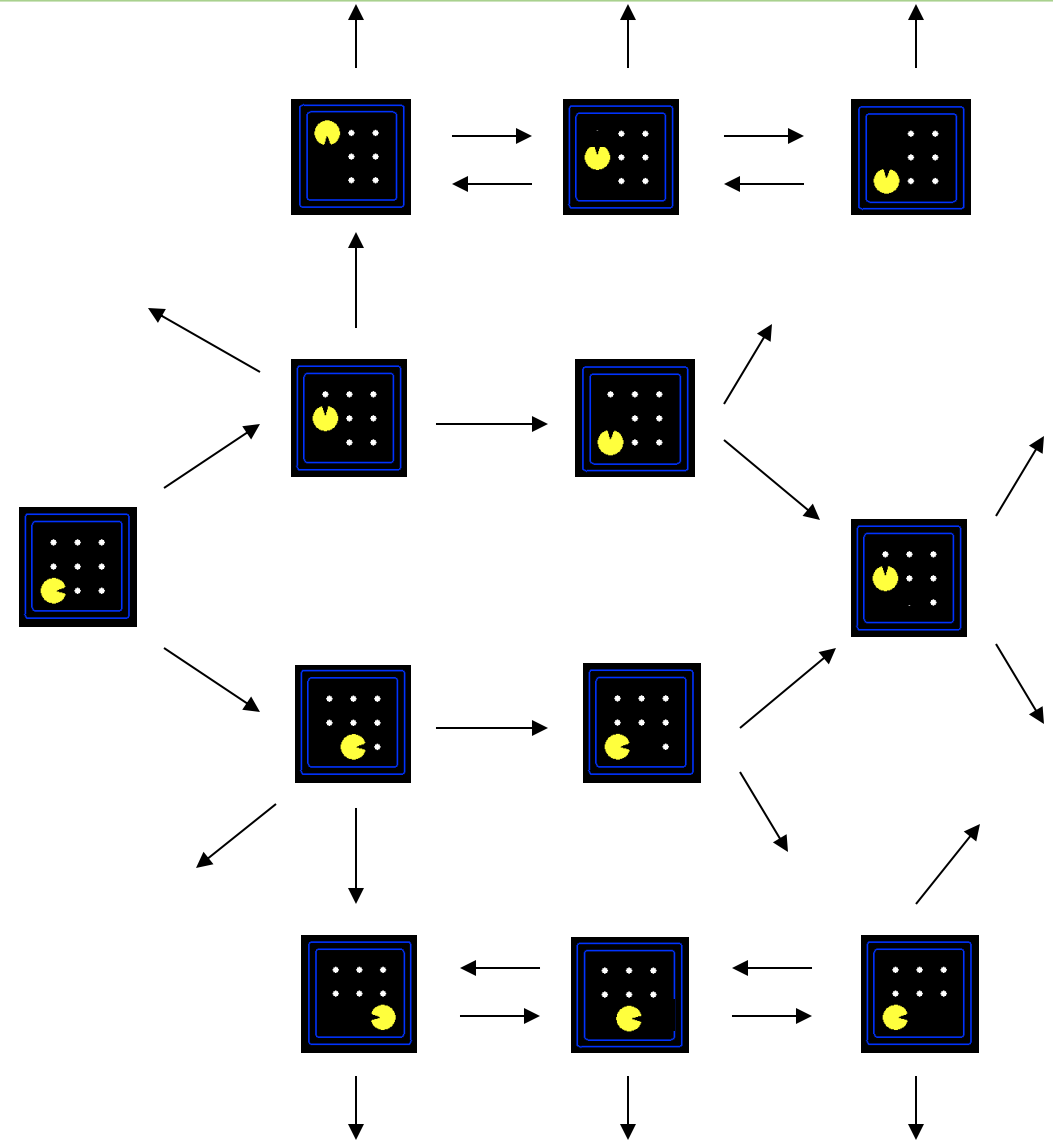
Example: Traveling in Romania



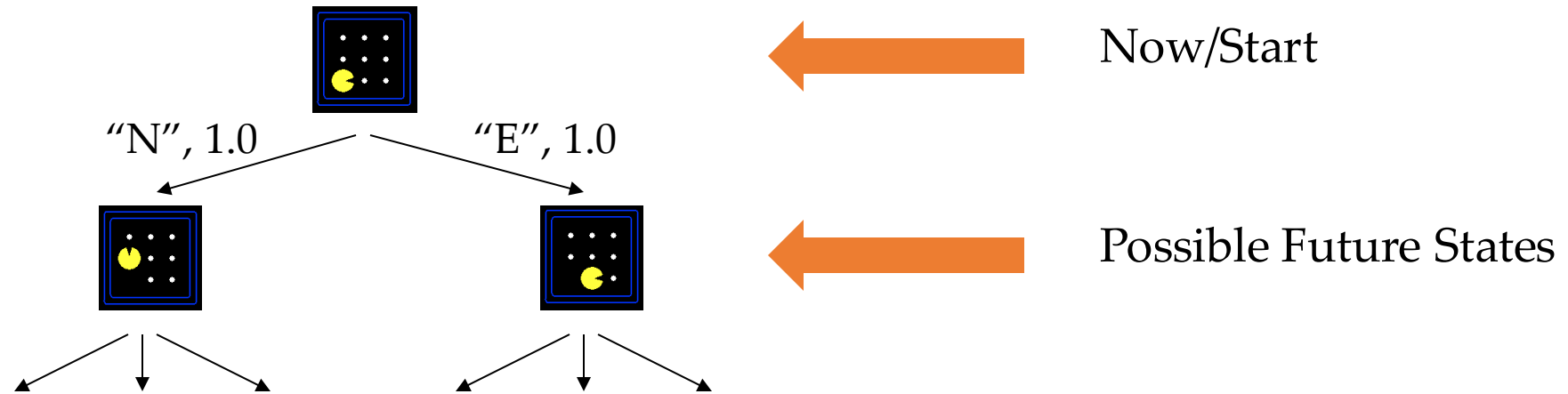
- State space:
 - Cities (location)
- Successor function:
 - Roads: Go to adjacent city with cost = distance
- Start state:
 - Arad
- Goal test:
 - Is state == Bucharest?

State Space Graphs

- State space graph: A mathematical representation of a search problem
 - Nodes are (abstracted) world configurations
 - Arcs represent successors (action results)
 - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



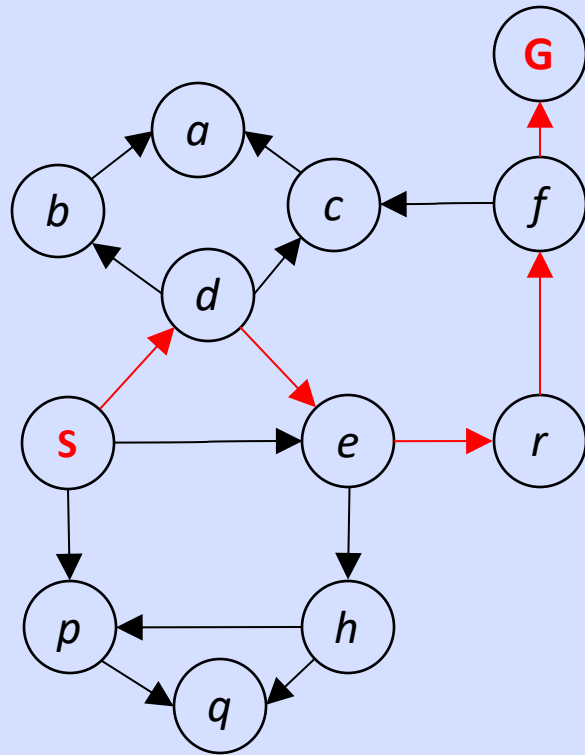
Search Trees



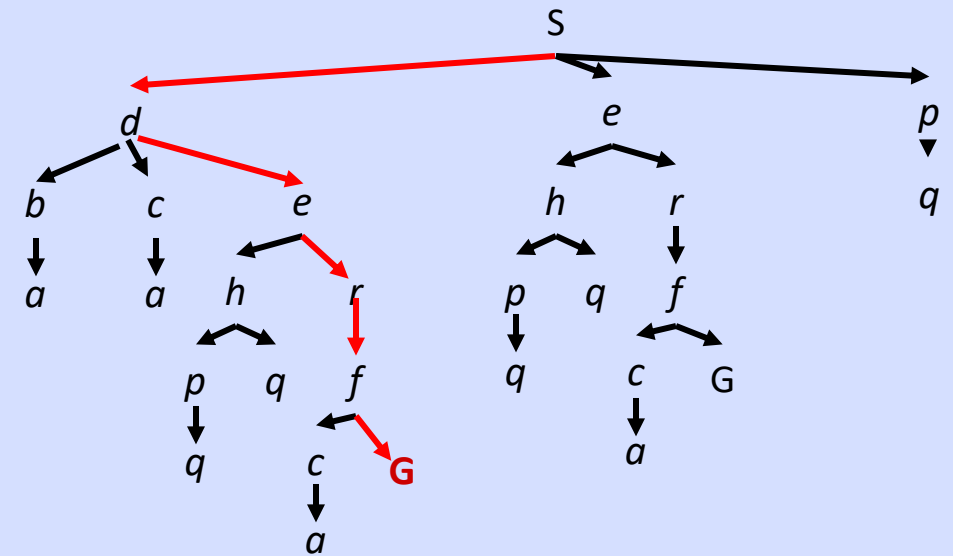
- A search tree:
 - A “what if” tree of plans and their outcomes
 - The start state is the root node
 - Children correspond to successors
 - Nodes show states, but correspond to plans(set of actions) that achieve those states
 - Building the whole tree is impossible for most problems

State Space Graphs vs. Search Trees

State Space Graph



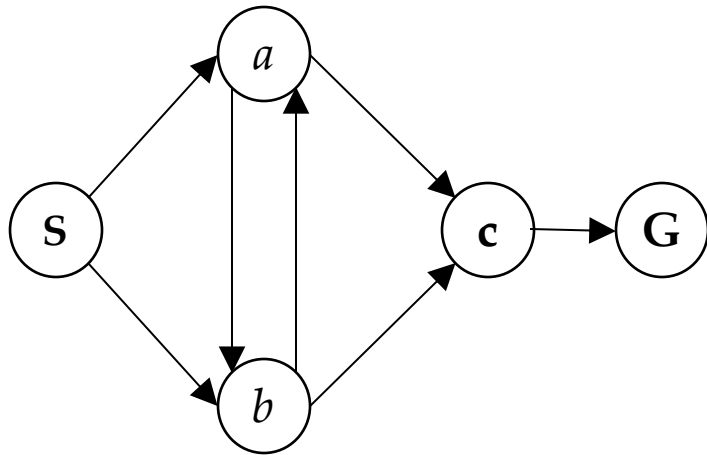
Search Tree



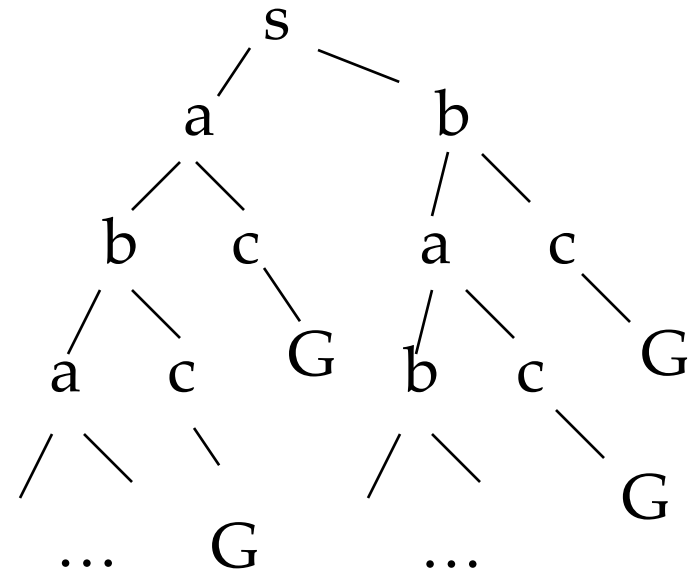
Each node in the search tree represents an ENTIRE PATH in the state space graph.

State Space Graphs vs. Search Trees

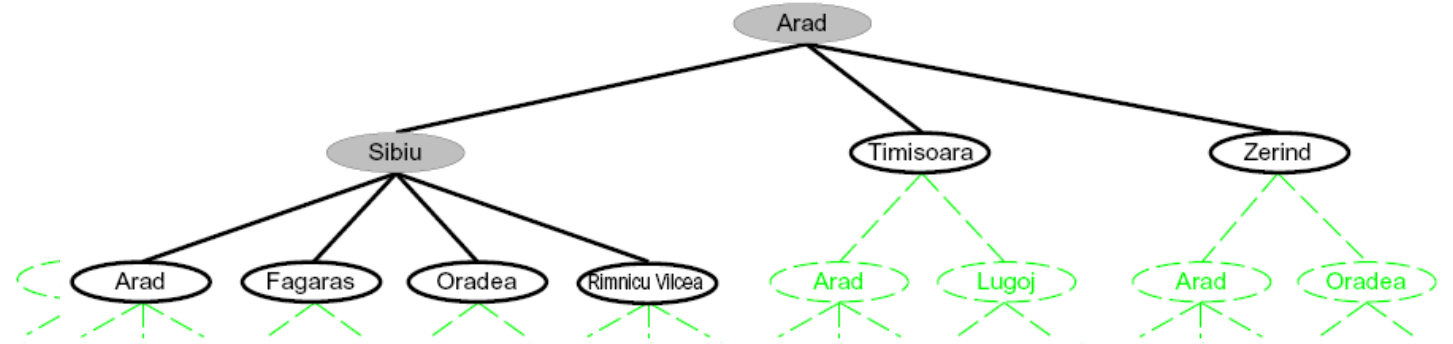
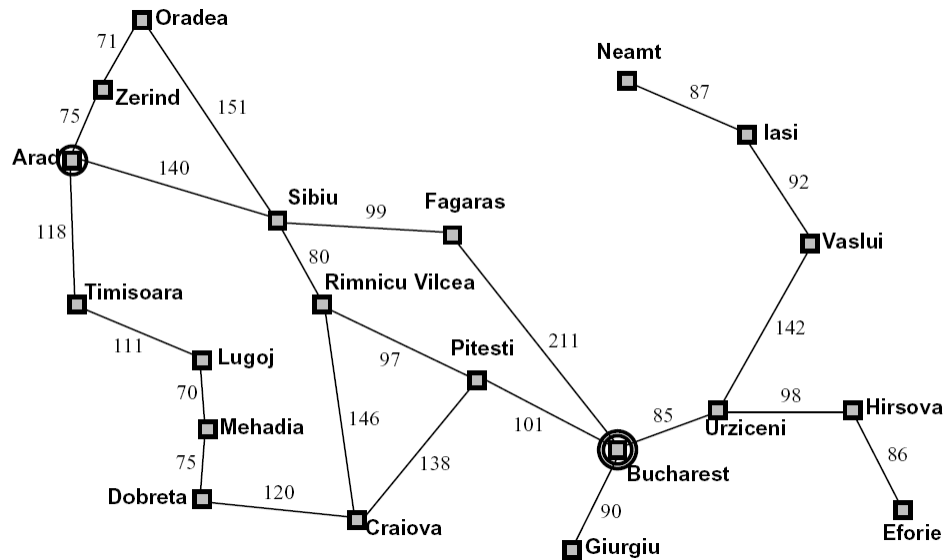
Consider this 5-state graph:



How big is its search tree (from S)?



Search Tree Example: Traveling in Romania



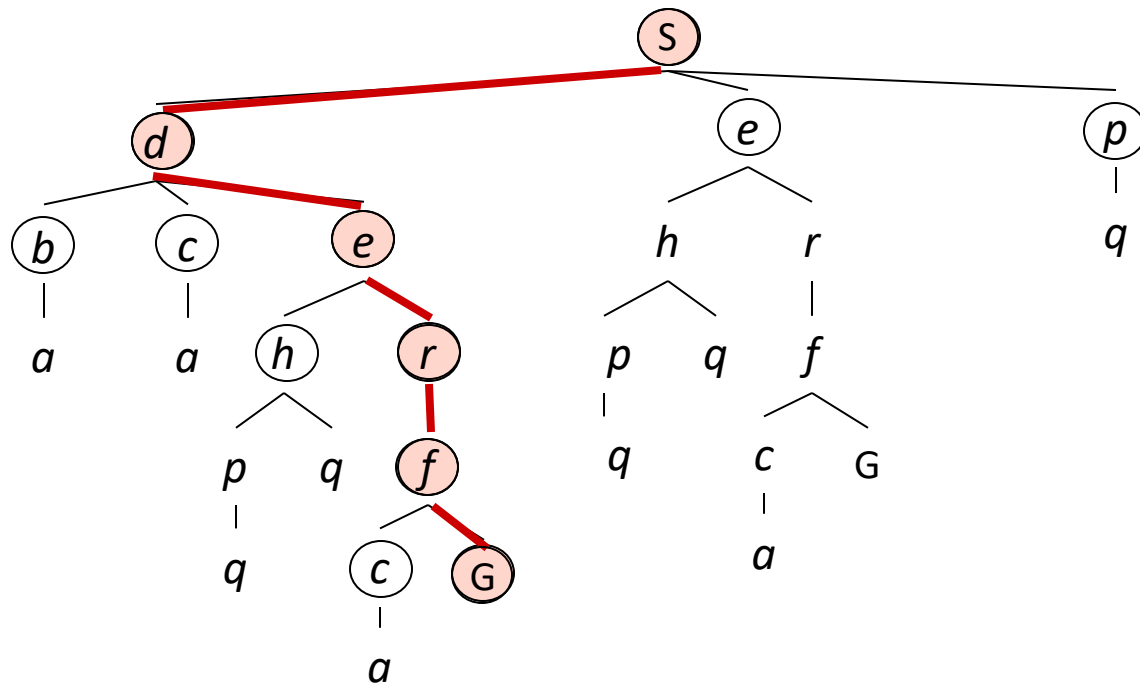
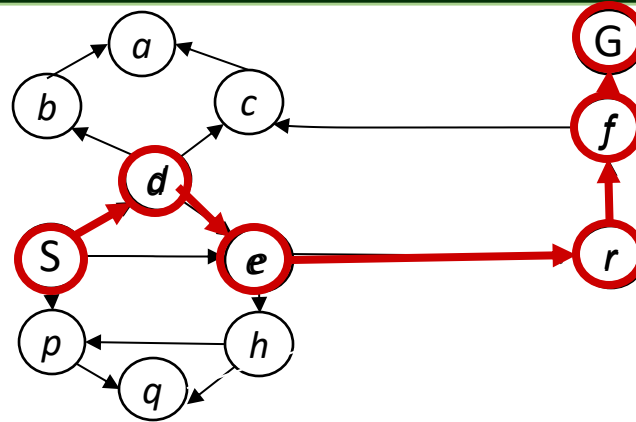
- Search:
 - Expand out potential plans (tree nodes)
 - Maintain a **fringe** of partial plans under consideration
 - Try to expand as few tree nodes as possible

General Tree Search

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

- Important ideas:
 - Fringe
 - Expansion
 - Exploration strategy

Example: Tree Search

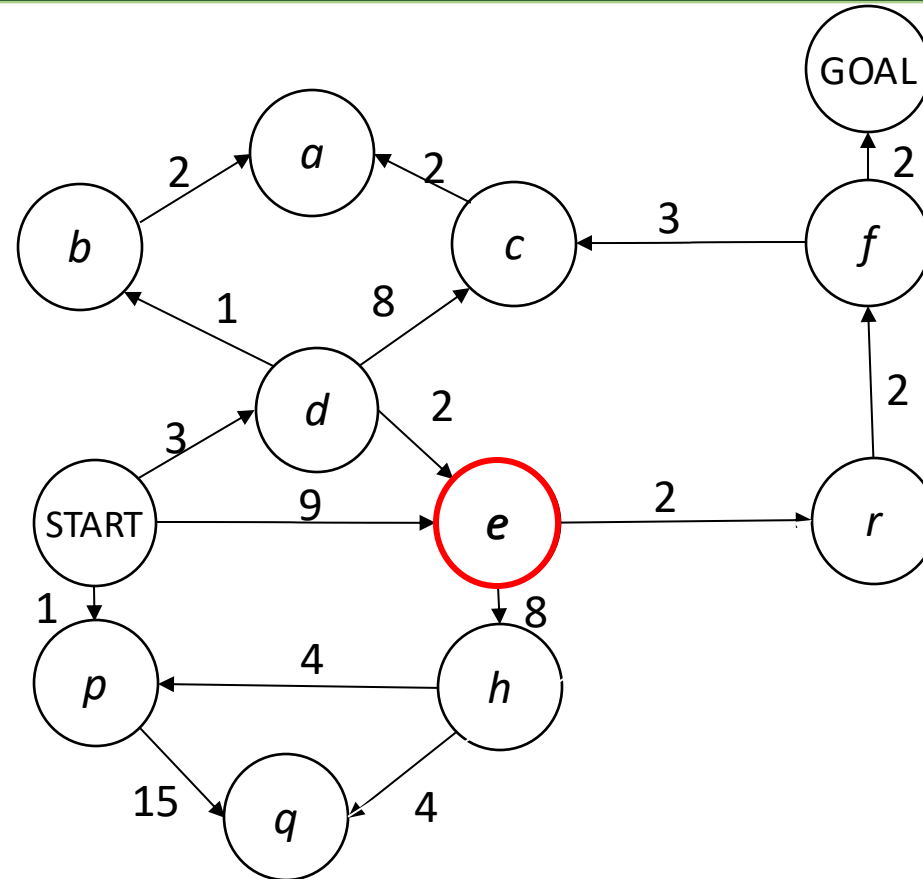


~~s~~
~~s → d~~
s → e
s → p
s → d → b
s → d → c
~~s → d → e~~
s → d → e → h
~~s → d → e → r~~
~~s → d → e → r → f~~
s → d → e → r → f → c
~~s → d → e → r → f → G~~

Outline

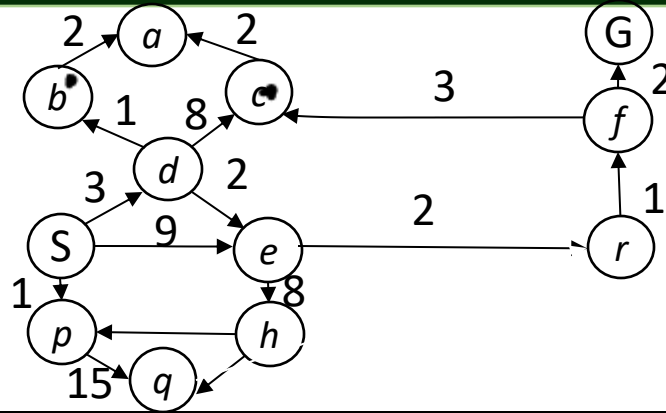
- Search Problems
- **Uninformed Search Methods**
 - Depth-First Search
 - Breadth-First Search
 - Uniform-Cost Search
- Heuristic Search
 - Greedy
 - A*

Cost-Sensitive Search



- BFS finds the shortest path in terms of number of actions.
- BFS does not find the least-cost path.
- We need an algorithm that finds the least-cost path.

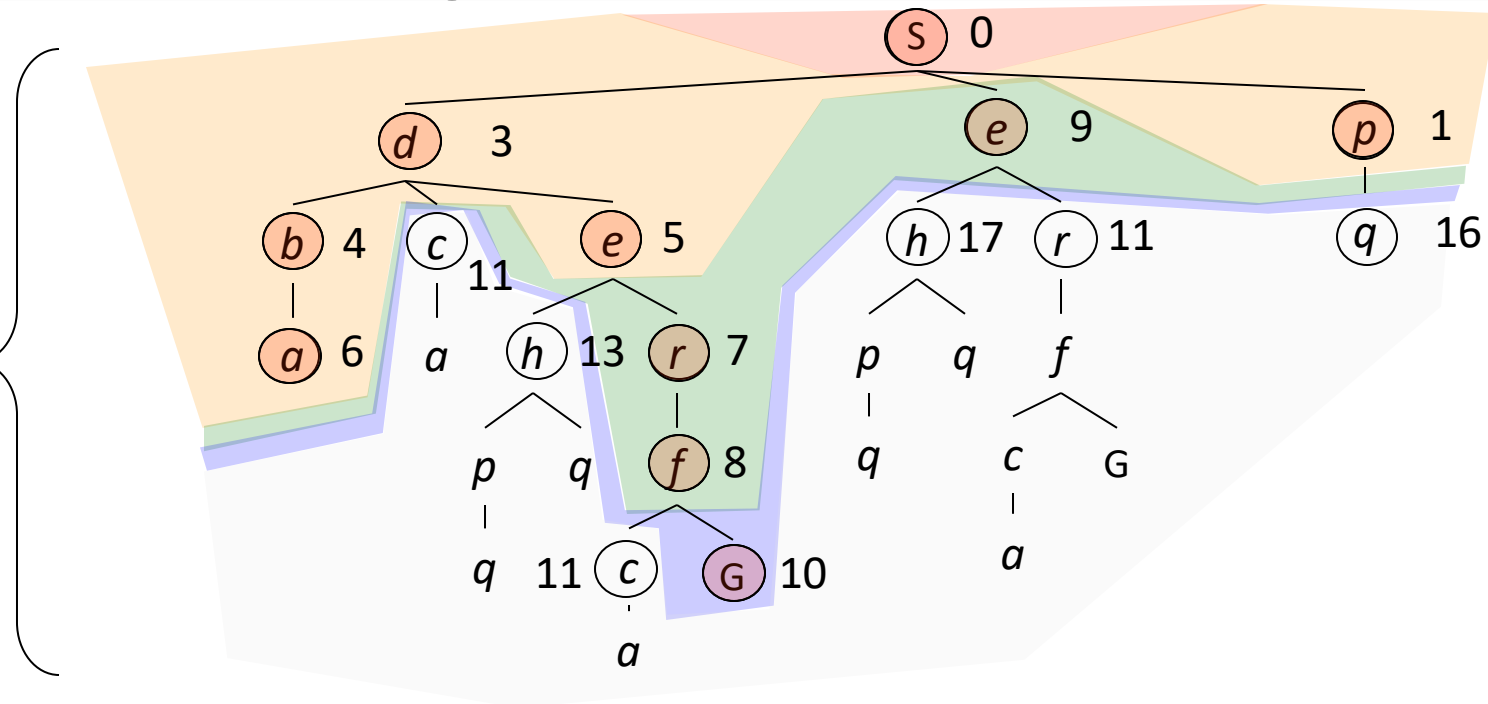
Uniform Cost Search



Strategy: expand a cheapest node first:

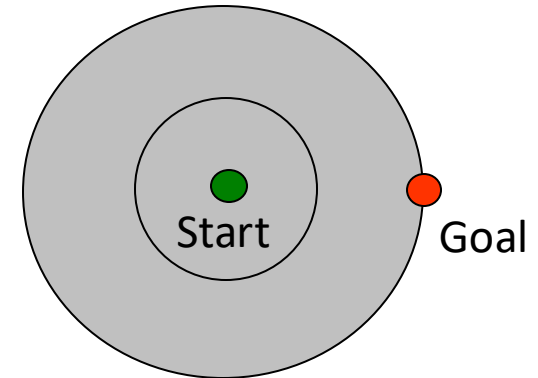
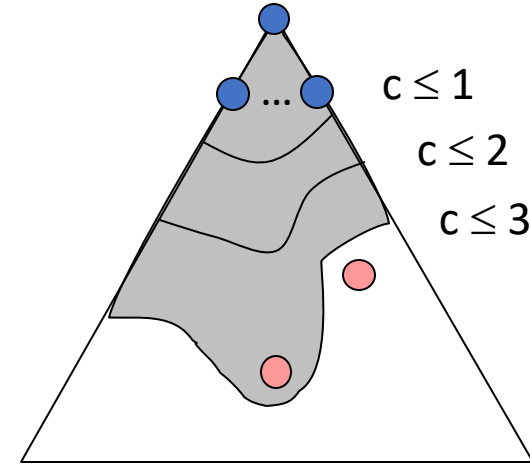
Implementation: fringe is a priority queue (priority: cumulative cost)

Cost contours

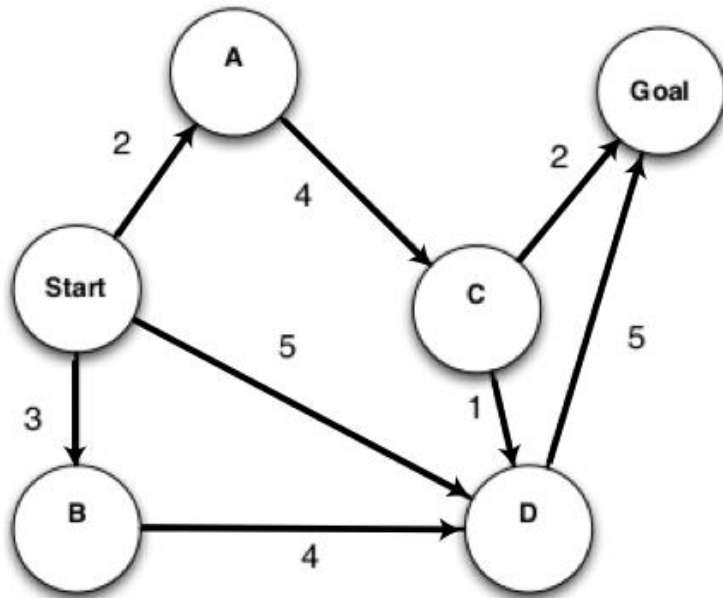


Uniform Cost Issues

- Remember: UCS explores increasing cost contours
- The good: UCS is complete and optimal!
 - Complete: Guaranteed to find a solution if one exists
 - Optimal: Guaranteed to find the least cost path
- The bad:
 - No information about goal location
 - Explores options in every “direction”



Example:



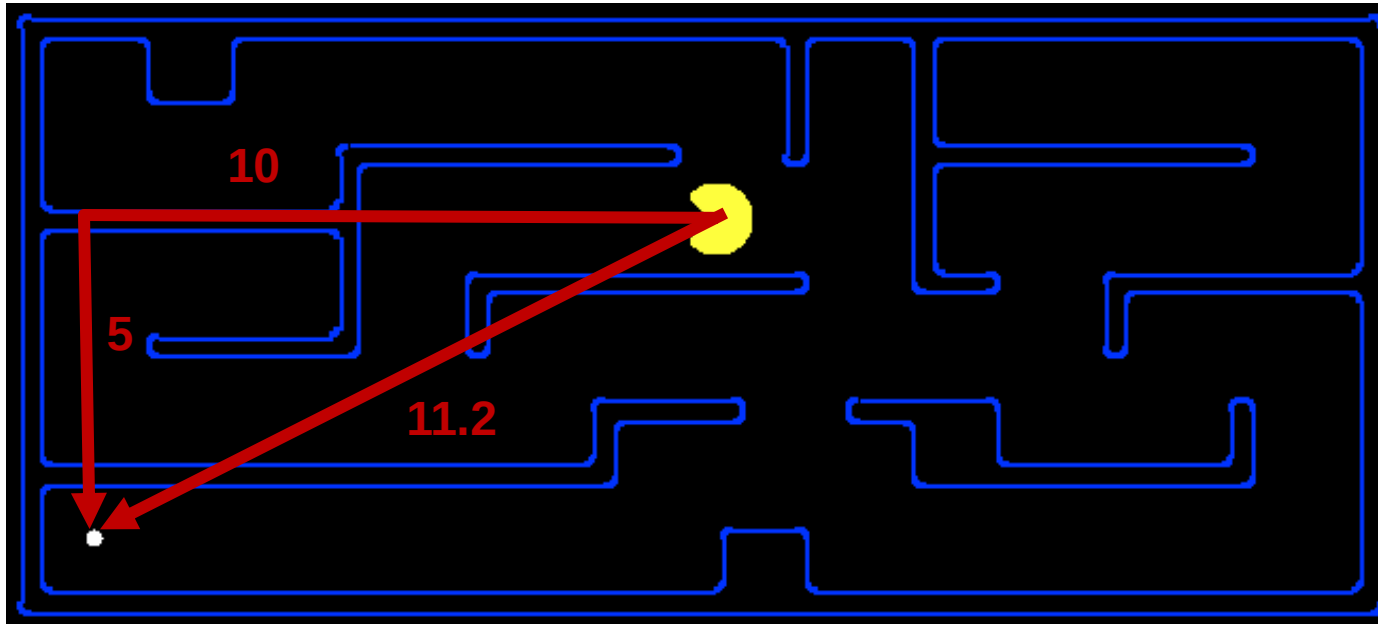
DFS:

BFS:

UCS:

Search Heuristics

- A heuristic is:
 - A function that estimates how close a state is to a goal
 - Designed for a particular search problem
 - Pathing?
 - Examples: Manhattan distance, Euclidean distance for pathing



Heuristic Function

$h(n)$: estimated cost of cheapest path from the state n to a goal

- Depends only on n (not path from root to n)
- If n is a goal state then $h(n)=0$
- $h(n) \geq 0$

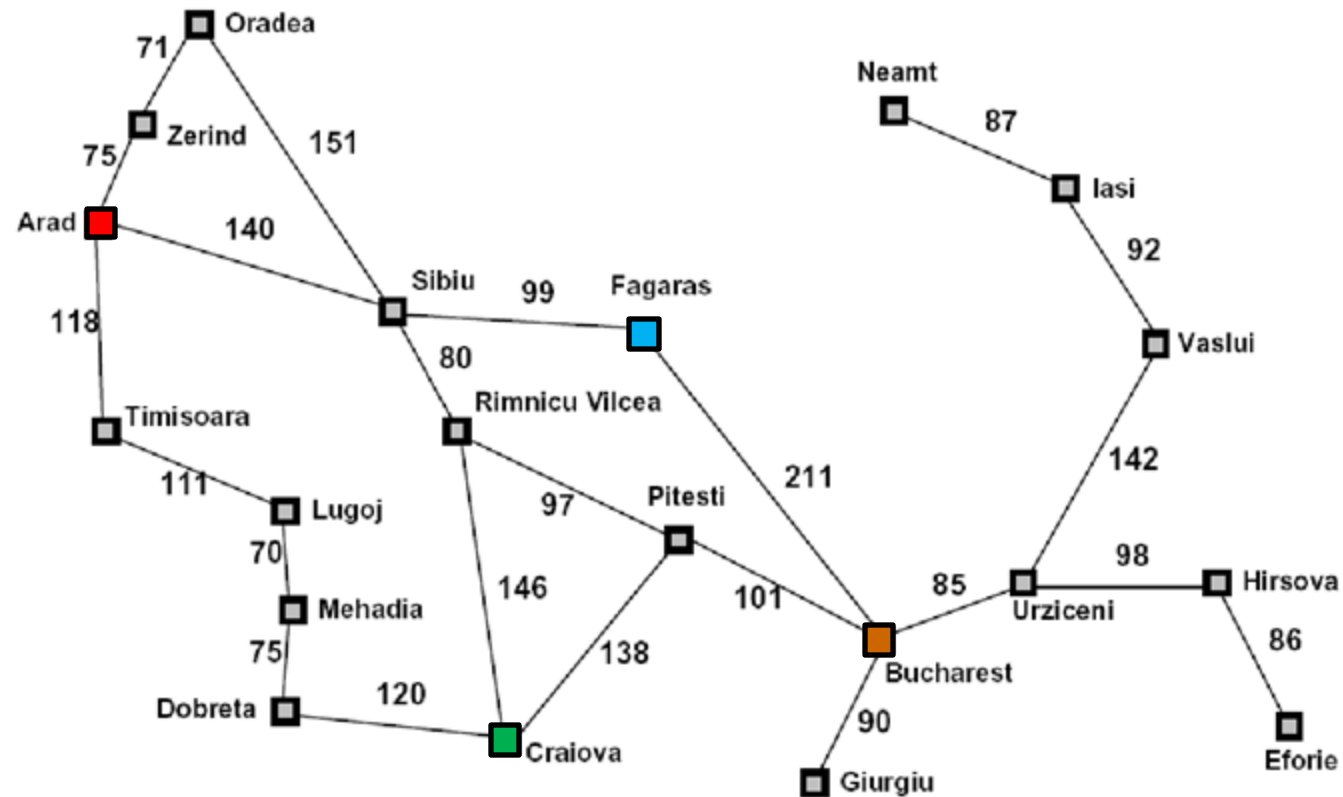
7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Example: Heuristic Function



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

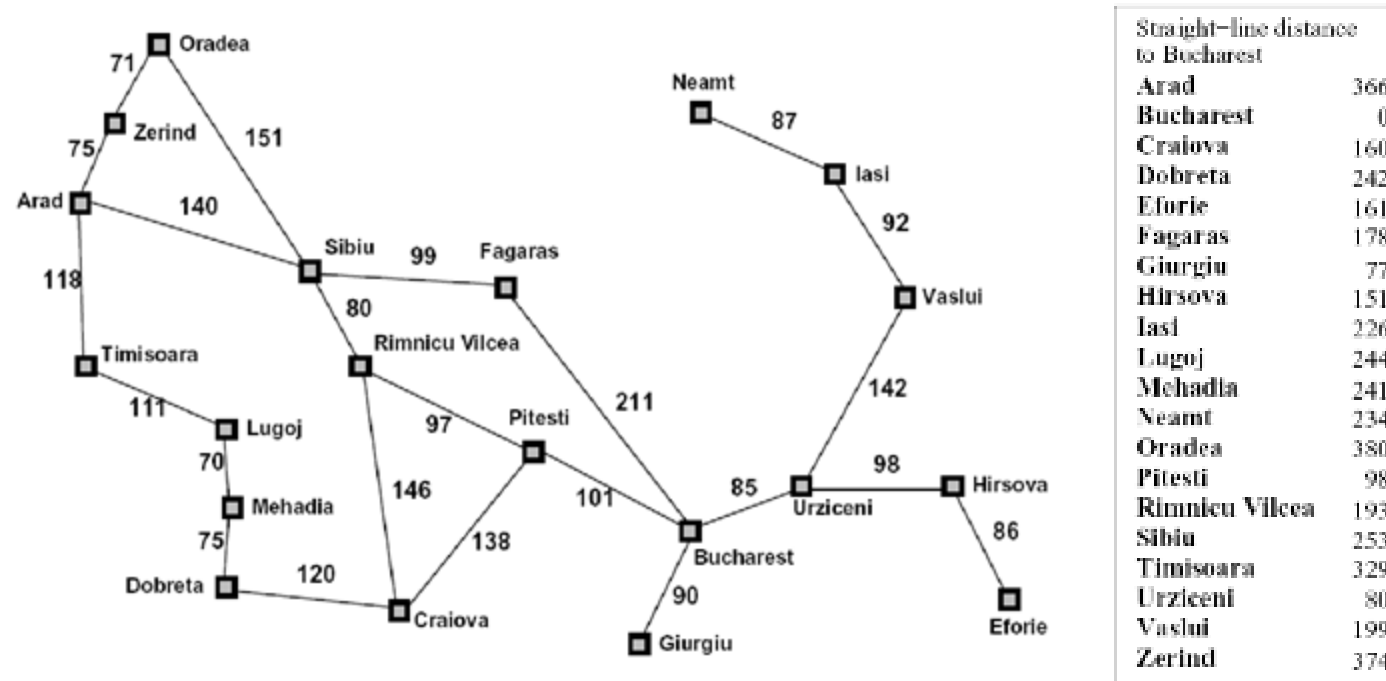
$h(x)$

Outline

- Search Heuristics
- **Uninformed Search Methods**
 - Depth-First Search
 - Breadth-First Search
 - Uniform-Cost Search
- **Heuristic Search Algorithm**
 - Greedy Search
 - A* Search

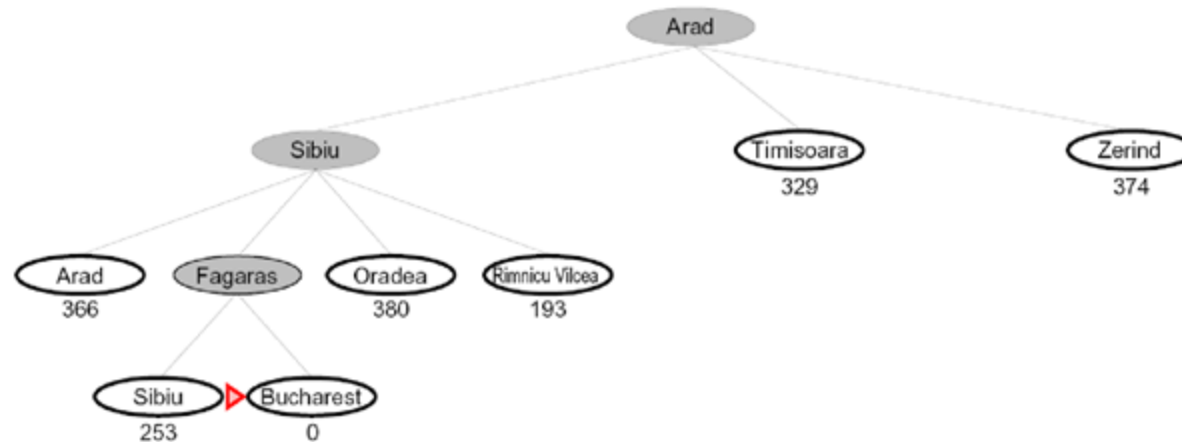
Greedy Search

- Expand the node that seems closest...



- Is it optimal?
 - No. Resulting path to Bucharest is not the shortest!

Greedy search

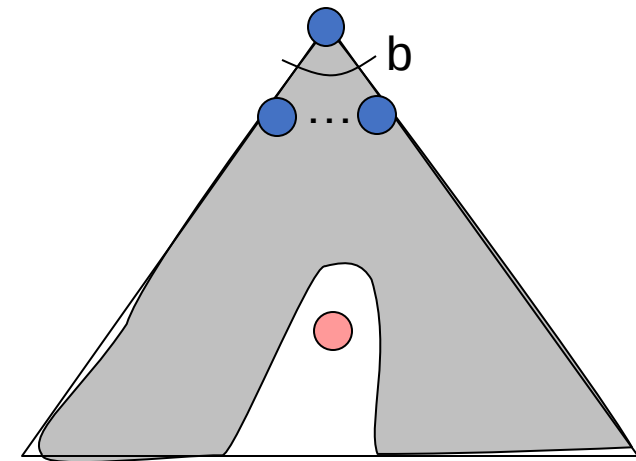
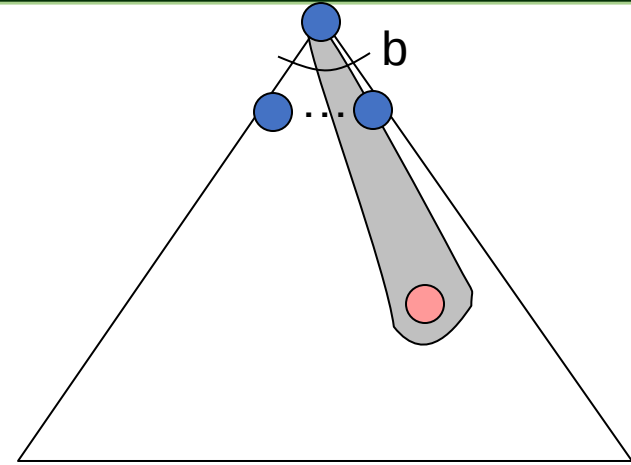


Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Oradea	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

- What can go wrong? Too aggressive!

Greedy Search

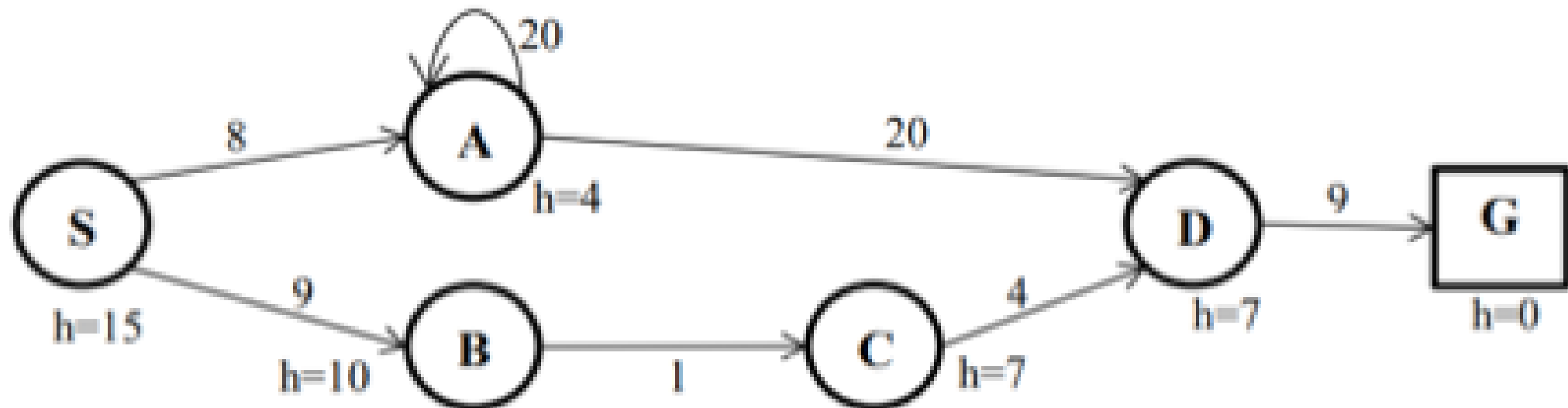
- Strategy: expand a node that you think is closest to a goal state
 - Heuristic: estimate of distance to nearest goal for each state
- A common case:
 - Best-first takes you straight to the (wrong) goal
- Worst-case: like a badly-guided DFS



Exercise

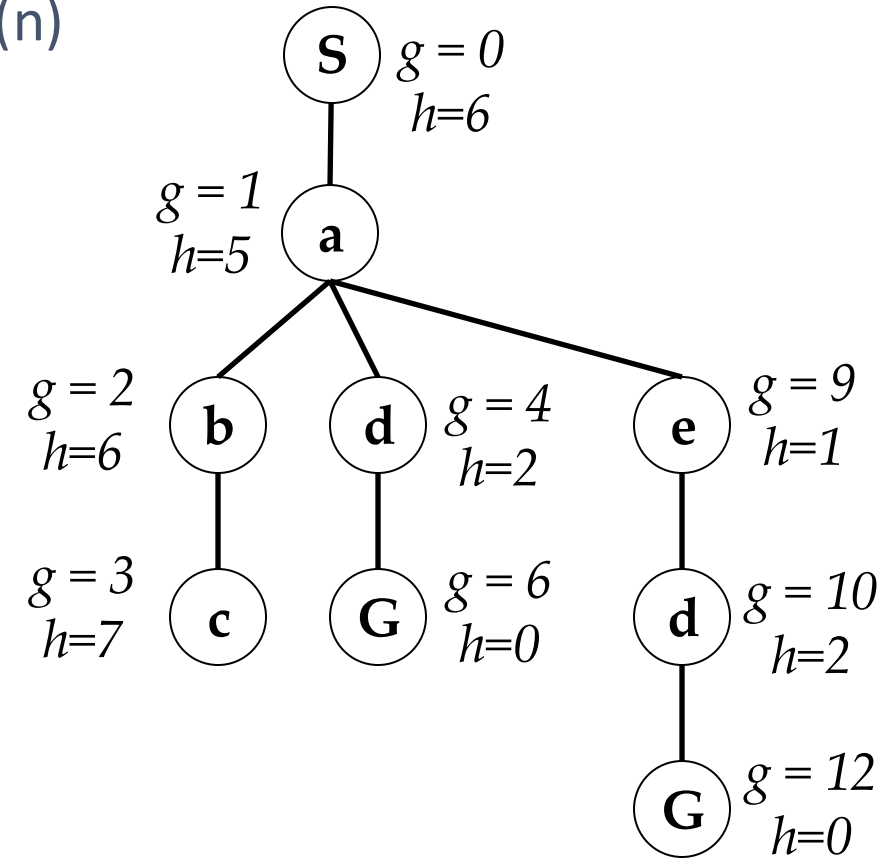
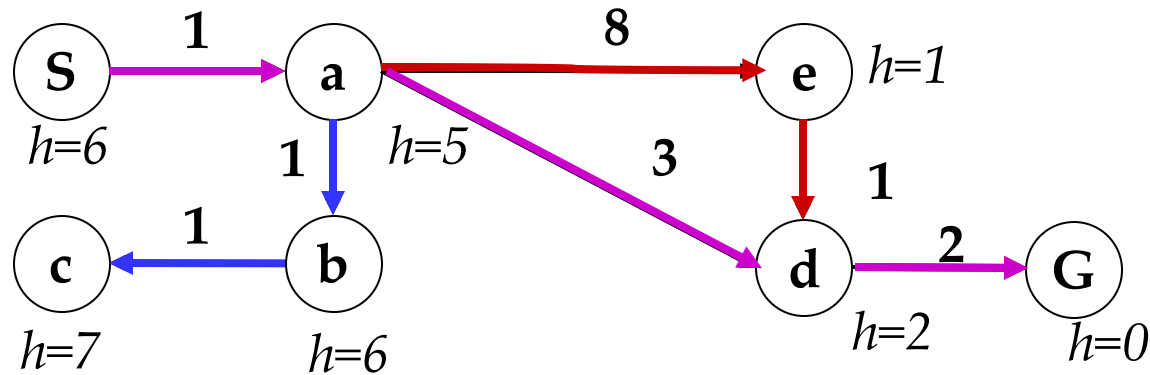
Q4. Execute Tree Search through this graph (i.e., do not remember visited nodes). Step costs are given next to each arc. Heuristic values are given next to each node (as $h=x$). The successors of each node are indicated by the arrows out of that node. Successors are returned in top-to-bottom order. Specifically, the children of S are (A, B) and the children of A are (A, D), in that order. For UNIFORM COST SEARCH, show the order in which nodes are expanded (i.e., to expand a node means that its children are generated), ending with the goal node that is found. (10 pts)

Answer:



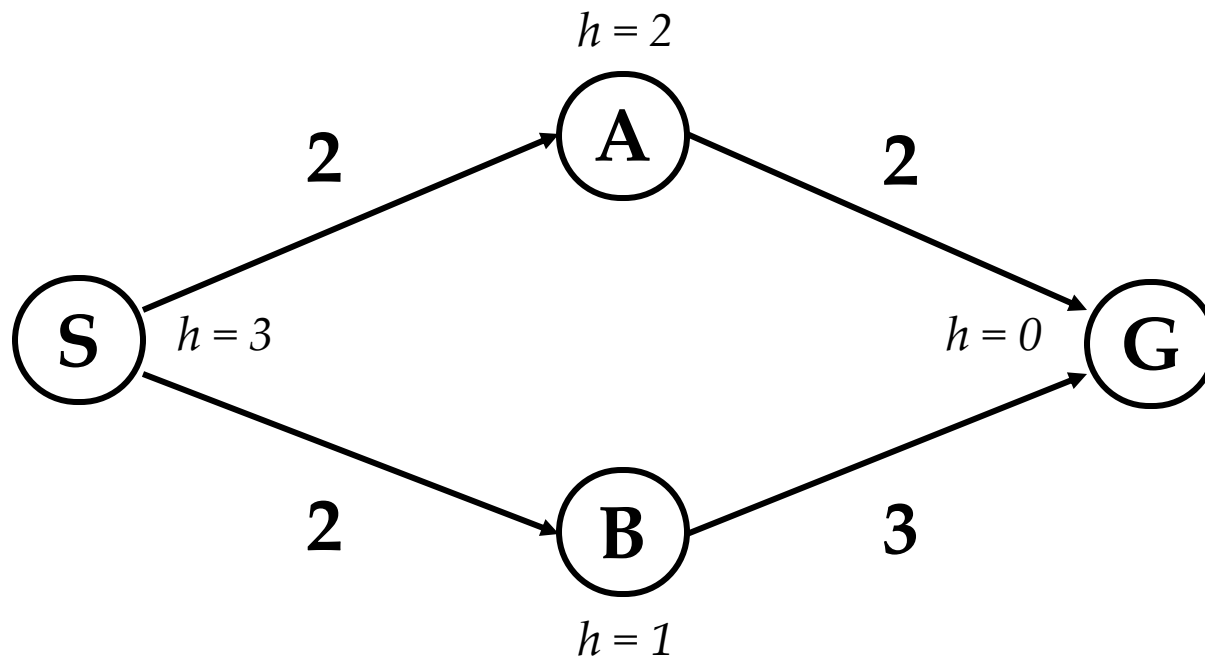
A* Search: Combining UCS and Greedy

- **Uniform-cost** orders by path cost, or *backward cost* $g(n)$
- **Greedy** orders by goal proximity, or *forward cost* $h(n)$
- **A* Search** orders by the sum: $f(n) = g(n) + h(n)$



When should A* terminate?

- Should we stop when we enqueue a goal?



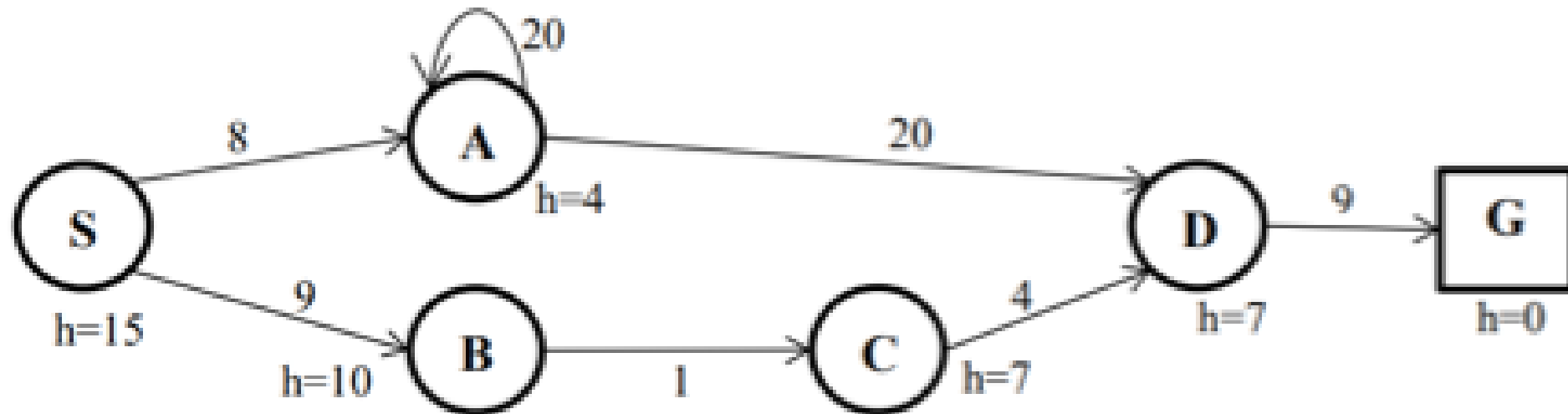
- No: only stop when we dequeue a goal

	g h +		
S	0	3	3
S->A	2	2	4
S->B	2	1	3
S->B->G	5	0	5
S->A->G	4	0	4

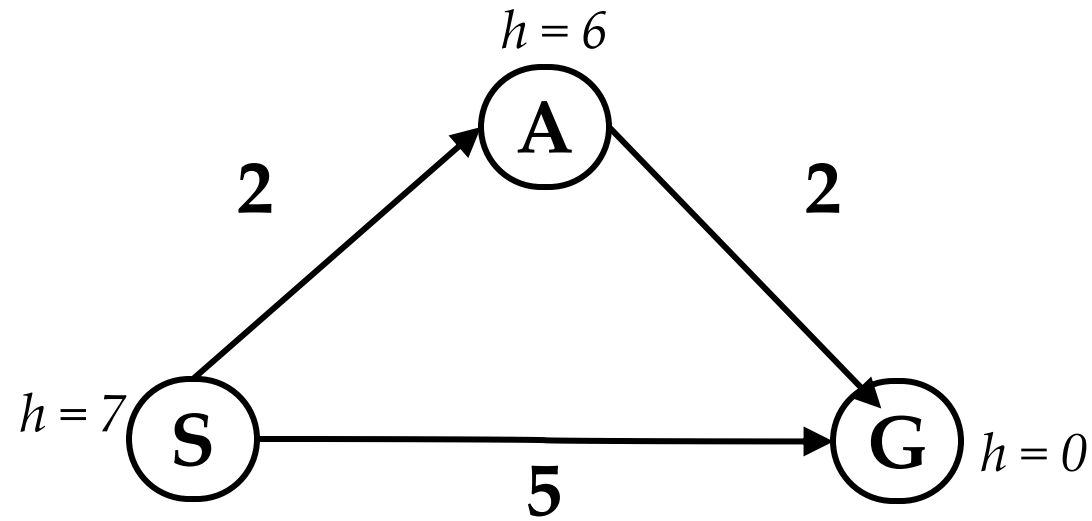
Exercise

Q5. Execute Tree Search through this graph (i.e., do not remember visited nodes). Step costs are given next to each arc. Heuristic values are given next to each node (as $h=x$). The successors of each node are indicated by the arrows out of that node. Successors are returned in top-to-bottom order. Specifically, the children of S are (A, B) and the children of A are (A, D), in that order. For A* Search, show the order in which nodes are expanded (i.e., to expand a node means that its children are generated), ending with the goal node that is found. (10 pts)

Answer:



Is A* Optimal?



- What went wrong?
- actual bad goal cost < estimated good goal cost
- We need estimates < actual costs

Admissibility

- Inadmissible (pessimistic) heuristics break optimality by trapping good plans on the fringe
- Admissible (optimistic) heuristics slow down bad plans but never outweigh true costs

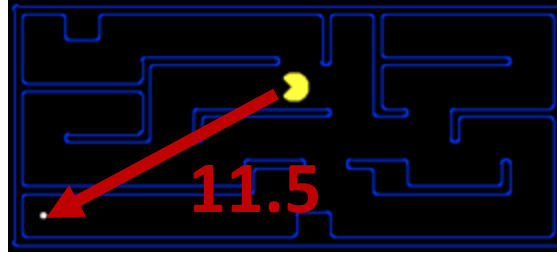
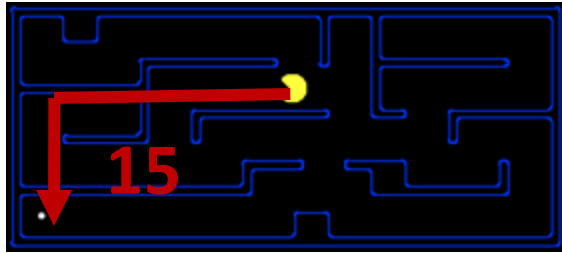
Admissible Heuristics

- A heuristic h is *admissible* (optimistic) if:

$$0 \leq h(n) \leq h^*(n)$$

where $h^*(n)$ is the true cost to a nearest goal

- Examples:

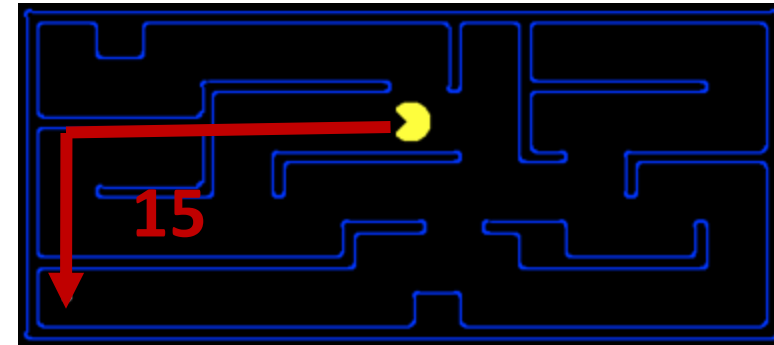


0.0

- In practice, coming up with admissible heuristics is most of what's involved in using A^* .

Creating Admissible Heuristics

- Most of the work in solving hard search problems optimally is in coming up with admissible heuristics
- Often, admissible heuristics are solutions to *relaxed problems*, where new actions are available



- Inadmissible heuristics are often useful too

Example: 8 Puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- What are the states?
- How many states?
- What are the actions?
- How many successors from the start state?
- What should the costs be?

Admissible
heuristics?

8 Puzzle I

- Heuristic: Number of tiles misplaced
- Why is it admissible?
- $h(\text{start}) = 8$
- This is a *relaxed-problem* heuristic

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Average nodes expanded vs optimal path			
	4 steps	8 steps	12 steps
UCS	112	6,300	3.6×10^6
TILES	13	39	227

8 Puzzle II

- What if we had an easier 8-puzzle where any tile could slide any direction at any time, ignoring other tiles?
- Total *Manhattan* distance
- Why is it admissible?
- $h(\text{start}) = 3 + 1 + 2 + \dots = 18$

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Average nodes expanded vs optimal path

	4 steps	8 steps	12 steps
TILES	13	39	227
MANHATTAN	12	25	73

Example

- If $h_1(n)$ is admissible and $h_2(n)$ is also admissible then, which one of the following answers is the best admissible heuristic?
 - $h_1 + h_2$
 - $\max(h_1, h_2)$
 - $\min(h_1, h_2)$
 - $h_1 - h_2$

8 Puzzle III

- How about using the *actual cost* as a heuristic?
 - Would it be admissible?
 - Would we save on nodes expanded?
 - What's wrong with it?
- With A*: a trade-off between quality of estimate and work per node
 - As heuristics get closer to the true cost, you will expand fewer nodes but usually do more work per node to compute the heuristic itself